

KUBERNETES ENTERPRISE CLUSTER IMPLEMENTATION USING VMWARE CLOUD DIRECTOR

Introduction

Kubernetes (K8s) is an open-source container orchestration platform designed to automate the deployment, scaling, and management of containerized applications. It enables organizations to efficiently manage applications across multiple servers while ensuring high availability, reliability, and optimal resource utilization. Kubernetes provides built-in capabilities such as automated deployment, load balancing, service discovery, self-healing, rolling updates, and persistent storage management, making it the industry standard for modern cloud-native application deployment.

In this project, an enterprise-level multi-node Kubernetes cluster is implemented using VMware Cloud Director. The environment consists of one Master Node and multiple Worker Nodes configured using **kubeadm**. The cluster demonstrates key Kubernetes functionalities, including application deployment, scaling, networking, service management, and fault tolerance, providing practical experience with a production-style container orchestration platform.

2. Importance of Kubernetes

Kubernetes has become the industry standard for container orchestration due to its ability to simplify application deployment and management in modern cloud environments. It improves application reliability, optimizes resource utilization, and ensures high availability through automated operations, making it an ideal platform for enterprise-scale deployments.

Key Benefits:

- High availability and fault tolerance
- Automatic scaling based on workload
- Efficient resource utilization
- Infrastructure automation
- Application portability across environments
- Support for cloud-native applications
- Simplified application deployment and management

3. Project Objectives

The primary objective of this project is to design and implement a production-style Kubernetes cluster on VMware Cloud Director while gaining practical experience in container orchestration and cluster management.

Objectives:

- Deploy a multi-node Kubernetes cluster on VMware Cloud Director
- Configure one Master Node and four Worker Nodes
- Deploy and manage containerized web applications
- Implement persistent storage for applications
- Deploy database services within the cluster

- Demonstrate Kubernetes self-healing and auto-recovery
- Perform rolling updates with minimal downtime
- Deploy MinIO object storage
- Configure the Kubernetes Dashboard for cluster management
- Monitor cluster resources and application health

4. Kubernetes Architecture

Kubernetes follows a **client–server architecture** that consists of a **Control Plane (Master Node)** and one or more **Worker Nodes**. The Control Plane is responsible for managing the entire cluster, while the Worker Nodes execute containerized applications. Users interact with the cluster through the Kubernetes API using tools such as **kubectl**. The Control Plane continuously monitors the cluster state and ensures that applications remain in the desired state by automatically scheduling workloads, recovering failed containers, and managing networking and storage. This architecture provides scalability, fault tolerance, high availability, and efficient resource utilization, making Kubernetes suitable for enterprise-grade deployments.

Kubernetes Architecture Overview

Component	Location	Description
kubectl	Client Machine	Command-line tool used to communicate with the Kubernetes cluster.
Kubernetes API Server (kube-apiserver)	Control Plane	Central communication point that processes all API requests and coordinates cluster operations.
etcd	Control Plane	Distributed key-value database that stores the complete cluster configuration and state.
Scheduler (kube-scheduler)	Control Plane	Selects the most suitable Worker Node for newly created Pods based on available resources and scheduling policies.
Controller Manager (kube-controller-manager)	Control Plane	Runs controllers that continuously monitor the cluster and maintain the desired state.
Cloud Controller Manager (Optional)	Control Plane	Integrates Kubernetes with cloud platforms such as AWS, Azure, GCP, or VMware Cloud Director.
Worker Node	Compute Node	Executes containerized workloads assigned by the Control Plane.
Kubelet	Worker Node	Node agent that communicates with the API Server and ensures Pods are running correctly.
Container Runtime	Worker Node	Software such as containerd or CRI-O responsible for running containers.
kube-proxy	Worker Node	Handles networking, load balancing, and communication between Pods and Services.

Pod	Worker Node	Smallest deployable unit in Kubernetes containing one or more containers.
Container	Inside Pod	Lightweight runtime environment that packages an application with its dependencies.

Table: Architecture Overview

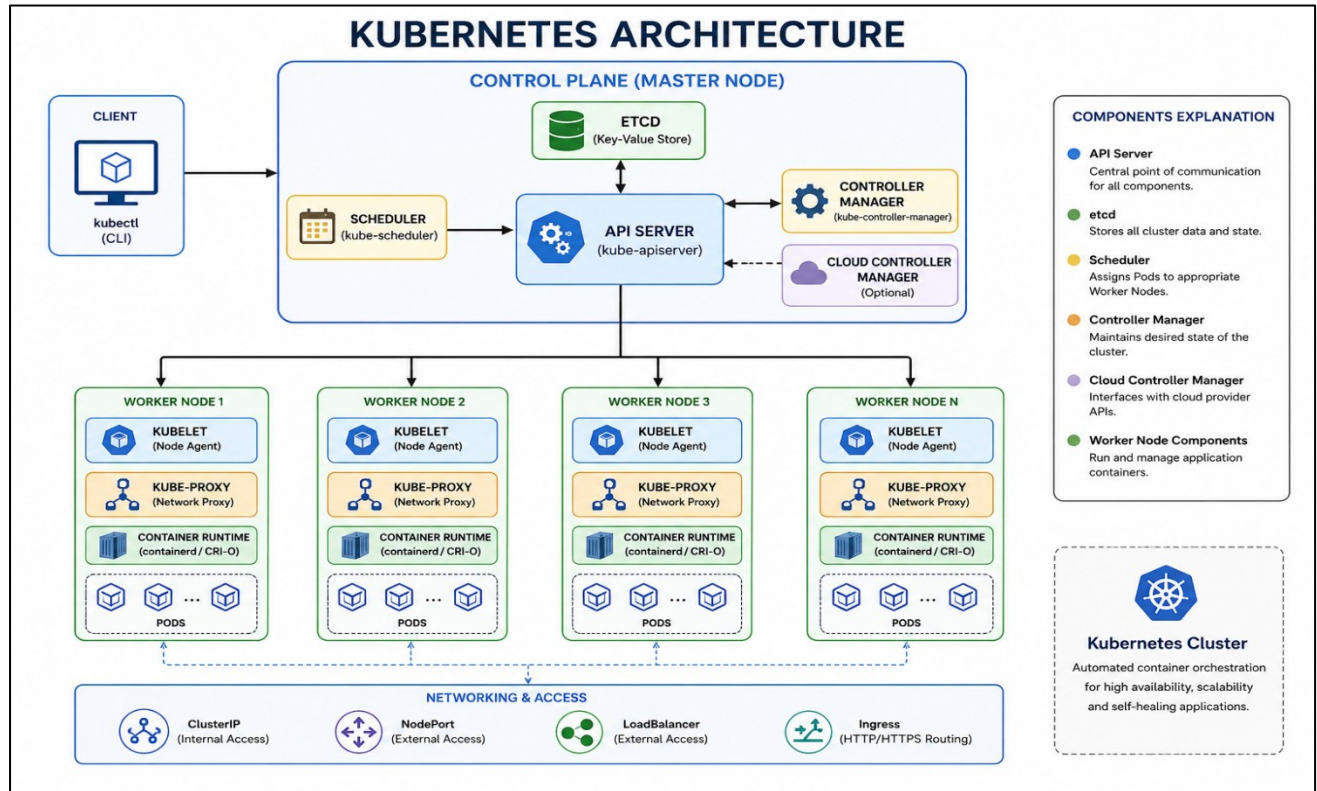


Figure: Kubernetes Architecture.

Common Kubernetes Terms

Term	Description
Cluster	Collection of Control Plane and Worker Nodes managed together.
Node	A physical or virtual machine participating in the Kubernetes cluster.
Master Node (Control Plane)	Manages scheduling, monitoring, networking, and overall cluster operations.
Worker Node	Runs application workloads inside Pods.
Pod	Smallest deployable object containing one or more containers.
Container	Packaged application with all required dependencies.
Image	Read-only template used to create containers (e.g., Nginx, MySQL).
Namespace	Provides logical isolation between resources within a cluster.
Deployment	Manages Pod creation, scaling, updates, and rollback.

ReplicaSet	Ensures the required number of Pod replicas are always running.
Service	Provides a stable network endpoint for accessing Pods.
ClusterIP	Internal service accessible only within the cluster.
NodePort	Exposes a service through a port on every Worker Node.
LoadBalancer	Exposes applications using an external load balancer.
Ingress	Manages HTTP/HTTPS routing to services using hostnames or paths.
ConfigMap	Stores non-sensitive configuration data separately from applications.
Secret	Securely stores sensitive information such as passwords and API keys.
Persistent Volume (PV)	Cluster storage resource independent of Pod lifecycle.
Persistent Volume Claim (PVC)	Request made by a Pod for persistent storage.
Storage Class	Defines dynamic storage provisioning policies.
DaemonSet	Ensures one Pod runs on every Worker Node.
StatefulSet	Manages stateful applications requiring stable identities and persistent storage.
Job	Executes a task once and exits successfully.
CronJob	Runs scheduled tasks periodically using cron expressions.
Label	Key-value pair used to organize and identify Kubernetes resources.
Selector	Selects resources based on matching labels.
Annotation	Stores additional metadata that is not used for resource selection.
Resource Quota	Limits resource consumption within a namespace.
Horizontal Pod Autoscaler (HPA)	Automatically increases or decreases Pod replicas based on CPU or memory usage.
kubectl	Command-line tool used to manage Kubernetes clusters.
YAML Manifest	Configuration file used to define Kubernetes resources declaratively.

Table: Common Terms

Architecture Workflow

- The user submits commands using **kubectl**.
- The **API Server** validates and processes the request.
- The desired cluster state is stored in **etcd**.
- The **Scheduler** selects the appropriate Worker Node.
- The **Kubelet** creates the required Pods using the container runtime.

- **kube-proxy** configures networking so applications can communicate.
- The **Controller Manager** continuously monitors the cluster and recreates failed Pods or scales applications when required.
- Users access applications through **Services**, **NodePort**, **LoadBalancer**, or **Ingress**.

System Specifications

Component	Specification
Platform	VMware Cloud Director
OS	Ubuntu 22.04 LTS
Kubernetes	v1.33
Container Runtime	containerd
Network Plugin	Calico
Dashboard	Kubernetes Dashboard
Storage	Persistent Volume
Object Storage	MinIO
Database	MySQL, PostgreSQL
Cache	Redis

Table: System specifications.

Virtual Machine Configuration

VM Name	Role	CPU	RAM	Disk
k8s-master	Control Plane	4	8 GB	60 GB
worker-1	Web Node	4	8 GB	50 GB
worker-2	Database Node	4	8 GB	50 GB
worker-3	Cache Node	4	8 GB	50 GB
worker-4	Storage Node	4	8 GB	50 GB

Table: Virtual Machine configurations.

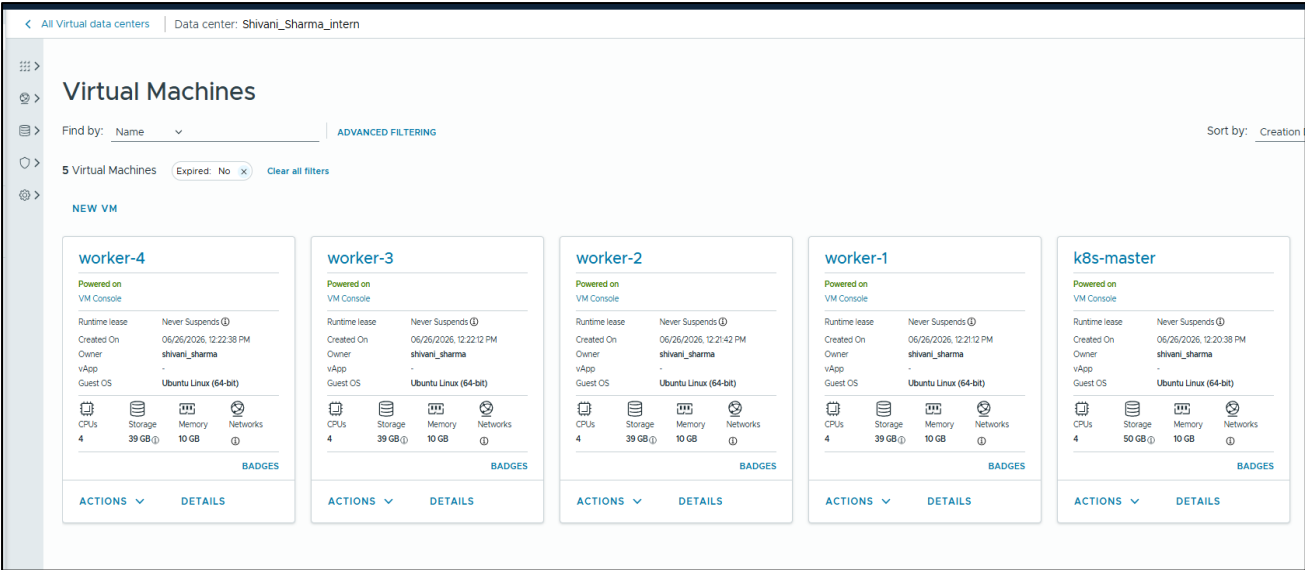


Figure: VMs Configuration.

Project Architecture

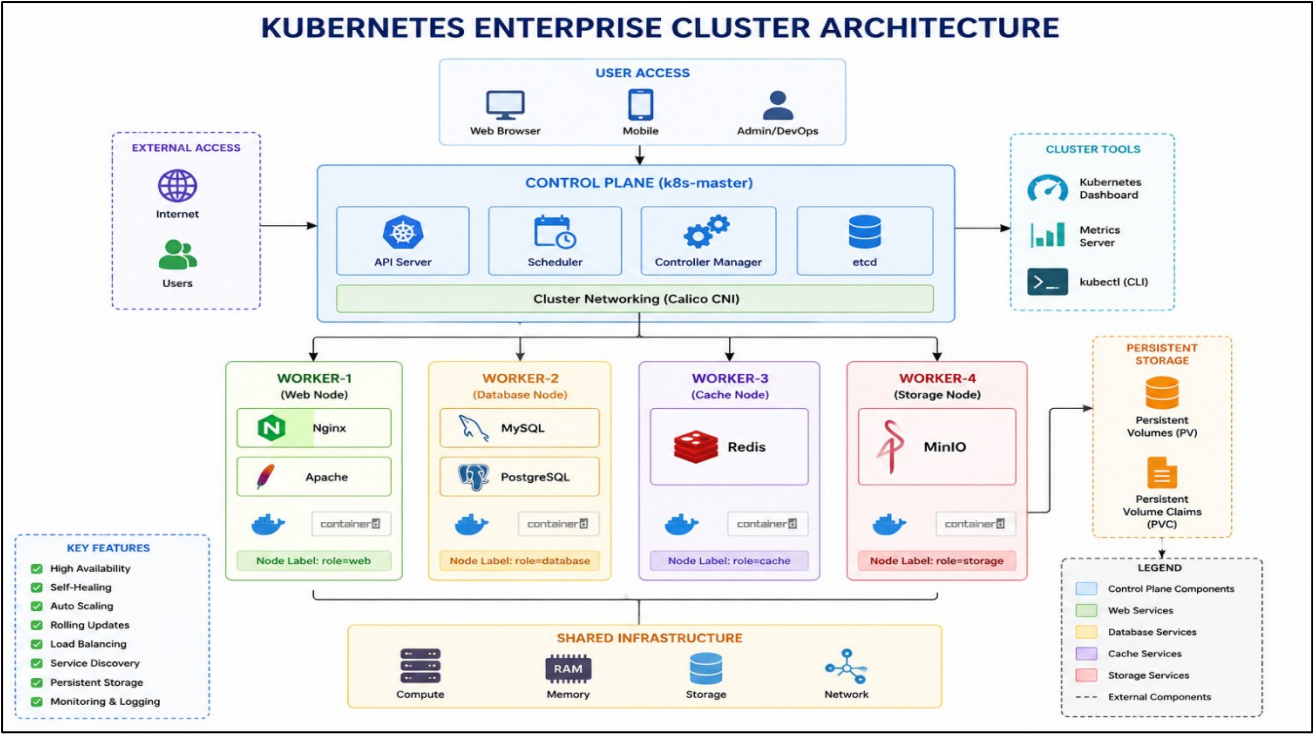


Figure: Project Architecture.

IMPLEMENTATION

System Update

Command	Purpose
apt update	Updates package repositories.
apt upgrade -y	Installs latest package updates.

Disable Swap

Command	Purpose
swapoff -a	Temporarily disables swap memory.
sed -i ' / swap / s/^/#' /etc/fstab	Permanently disables swap after reboot.
free -h	Verifies swap has been disabled.

```
root@k8s-master:~# swapoff -a
sed -i '/swap/d' /etc/fstab
root@k8s-master:~# free -h
              total        used        free      shared  buff/cache   available
Mem:           9.7Gi        376Mi        4.8Gi        1.0Mi        4.5Gi        9.0Gi
Swap:            0B           0B           0B
```

Figure: Disable swap

4.3 Enable Kernel Modules

Command	Purpose
modprobe overlay	Loads overlay filesystem module.
modprobe br_netfilter	Enables bridge networking module.
cat <<EOF > /etc/modules-load.d/k8s.conf	Makes modules persistent after reboot.

```
root@k8s-master:~# cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
overlay
br_netfilter
root@k8s-master:~# modprobe overlay
modprobe br_netfilter
root@k8s-master:~#
```

Figure: Enable Kernel Modules

Configure Network Parameters

Command	Purpose
cat <<EOF > /etc/sysctl.d/k8s.conf	Configures Kubernetes networking.
sysctl --system	Applies network configuration.

```

root@k8s-master:~# cat <<EOF | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
root@k8s-master:~# sysctl --system
* Applying /etc/sysctl.d/10-console-messages.conf ...
kernel.printk = 4 4 1 7
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
kernel.kptr_restrict = 1
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
kernel.sysrq = 176
* Applying /etc/sysctl.d/10-network-security.conf ...
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
* Applying /etc/sysctl.d/10-ptrace.conf ...
kernel.yama.ptrace_scope = 1
* Applying /etc/sysctl.d/10-zero-page.conf ...
vm.mmap_min_addr = 65536
* Applying /usr/lib/sysctl.d/50-default.conf ...
kernel.core_uses_pid = 1

```

Figure: Configure Network Parameters

Install Container Runtime

Command	Purpose
apt install containerd -y	Installs container runtime.
mkdir -p /etc/containerd	Creates configuration directory.
containerd config default > /etc/containerd/config.toml	Generates default configuration.
systemctl restart containerd	Restarts container runtime.
systemctl enable containerd	Starts containerd automatically after reboot.

```

root@k8s-master:~# mkdir -p /etc/containerd
root@k8s-master:~# containerd config default > /etc/containerd/config.toml
root@k8s-master:~# nano /etc/containerd/config.toml
root@k8s-master:~# systemctl restart containerd
systemctl enable containerd
root@k8s-master:~# systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/lib/systemd/system/containerd.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 07:23:52 UTC; 11s ago
     Docs: https://containerd.io
   Main PID: 54631 (containerd)
    Tasks: 9
   Memory: 14.5M
      CPU: 72ms
   CGroup: /system.slice/containerd.service
           └─54631 /usr/bin/containerd

```

Figure: Install Container Runtime

Install Kubernetes Packages

Command	Purpose
apt install apt-transport-https ca-certificates curl gpg -y	Installs required dependencies.
mkdir -p /etc/apt/keyrings	Creates keyring directory.
curl -fsSL ...	Downloads Kubernetes GPG key.
echo "deb ..."	Adds Kubernetes repository.
apt update	Refreshes package index.
apt install kubelet kubeadm kubectl -y	Installs Kubernetes components.
apt-mark hold kubelet kubeadm kubectl	Prevents automatic updates.

```
root@k8s-master:~# mkdir -p /etc/apt/keyrings
root@k8s-master:~# curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | gpg --dearmor -o /etc/apt/keyring
root@k8s-master:~# echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v
.d/kubernetes.list
root@k8s-master:~# apt update
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Get:1 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb InRelease [1,192 B]
Get:4 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb Packages [20.3 kB]
Hit:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-security InRelease
Fetched 21.5 kB in 2s (12.5 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
root@k8s-master:~# apt install kubelet kubeadm kubectl -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0 libqmi-glib5 libqmi-proxy libse
  tcl tcl8.6 usb-modeswitch usb-modeswitch-data
Use 'apt autoremove' to remove them.
The following additional packages will be installed:
  conntrack cri-tools kubernetes-cni
The following NEW packages will be installed:
  conntrack cri-tools kubeadm kubectl kubelet kubernetes-cni
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 93.8 MB of archives.
After this operation, 343 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 conntrack amd64 1:1.4.6-2build2 [33.5 kB]
Get:2 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb cri-tools 1.30.1-1.1 [21.3 M
Get:3 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb kubeadm 1.30.14-1.1 [10.5 M
Get:4 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb kubectl 1.30.14-1.1 [10.9 M
Get:5 https://prod-cdn.packages.k8s.io/repositories/iscv:/kubernetes:/core:/stable:/v1.30/deb kubelet 1.30.14-1.1 [10.9 M
```

Figure: Install Kubernetes Packages

MASTER NODE CONFIGURATION

Initialize Control Plane

Command	Purpose
kubeadm init --pod-network-cidr=192.168.0.0/16	Initializes Kubernetes master node.

```
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.0.2:6443 --token ms6gt0.d41n33gxhc5u4364 \
--discovery-token-ca-cert-hash sha256:2b546d251c51b9463c5895f6e6c87bdae939319e6cc6d0a1679739789b362e35
```

Figure: Initialize Control Plane

Configure Kubectl

Command	Purpose
<code>mkdir -p \$HOME/.kube</code>	Creates kube directory.
<code>cp /etc/kubernetes/admin.conf \$HOME/.kube/config</code>	Copies cluster configuration.

NETWORK PLUGIN INSTALLATION

Install Calico

Command	Purpose
<code>kubectl apply -f calico.yaml</code>	Installs Calico CNI plugin.
<code>kubectl get pods -n kube-system</code>	Verifies Calico installation.

JOIN WORKER NODES

Command	Purpose
<code>kubeadm join MASTER-IP:6443 --token TOKEN</code>	Adds worker node to cluster.
<code>kubectl get nodes</code>	Verifies connected nodes.

```

root@worker-1:~# kubeadm join 192.168.0.2:6443 \
--token ms6gt0.d4ln33gxhc5u4364 \
--discovery-token-ca-cert-hash sha256:2b546d251c51b9463c5895f6e6c87bdae939319e6cc6d0a1679739789b362e35
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.823731ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@worker-1:~#

```

```

root@k8s-master:~# kubectl get nodes
NAME             STATUS    ROLES    AGE      VERSION
k8s-master       Ready     control-plane  8m53s    v1.30.14
worker-1         Ready     <none>      33s      v1.30.14
worker-2         Ready     <none>      102s     v1.30.14
worker-3         Ready     <none>      2m43s    v1.30.14
worker-4         Ready     <none>      6m3s     v1.30.14
root@k8s-master:~#

```

```

root@k8s-master:~# kubectl get pods -A
NAMESPACE      NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system    calico-kube-controllers-5b64744bcd-65dd7              1/1     Running   0          20m
kube-system    calico-node-4cp69                                      1/1     Running   0          13m
kube-system    calico-node-54stq                                      1/1     Running   0          12m
kube-system    calico-node-7kfjq                                      1/1     Running   0          20m
kube-system    calico-node-md7r5                                       1/1     Running   0          14m
kube-system    calico-node-x9hz7                                       1/1     Running   0          18m
kube-system    coredns-55cb58b774-kcsn8                              1/1     Running   0          20m
kube-system    coredns-55cb58b774-s6tv7                              1/1     Running   0          20m
kube-system    etcd-k8s-master                                         1/1     Running   0          20m
kube-system    kube-apiserver-k8s-master                             1/1     Running   0          20m
kube-system    kube-controller-manager-k8s-master                    1/1     Running   0          20m
kube-system    kube-proxy-2nj2w                                        1/1     Running   0          12m
kube-system    kube-proxy-c4k57                                       1/1     Running   0          13m
kube-system    kube-proxy-hnnmb                                       1/1     Running   0          20m
kube-system    kube-proxy-pmk9g                                       1/1     Running   0          14m
kube-system    kube-proxy-wvtmq                                       1/1     Running   0          18m
kube-system    kube-scheduler-k8s-master                             1/1     Running   0          20m
root@k8s-master:~#

```

Figure: Join worker nodes

Node Labeling

Command	Purpose
kubectl label node worker-1 role=web	Assigns Web node role.
kubectl label node worker-2 role=database	Assigns Database node role.
kubectl label node worker-3 role=cache	Assigns Cache node role.
kubectl label node worker-4 role=storage	Assigns Storage node role.

```

root@k8s-master:~# kubectl label node worker-1 role=web
kubectl label node worker-2 role=database
kubectl label node worker-3 role=storage
kubectl label node worker-4 role=apps
node/worker-1 labeled
node/worker-2 labeled
node/worker-3 labeled
node/worker-4 labeled
root@k8s-master:~# kubectl get nodes --show-labels
NAME                STATUS    ROLES    AGE   VERSION   LABELS
k8s-master          Ready     control-plane   22m   v1.30.14   beta.kubernetes.io/arch=amd64,
k8s-master, kubernetes.io/os=linux, node-role.kubernetes.io/control-plane=, node
worker-1            Ready     <none>    14m   v1.30.14   beta.kubernetes.io/arch=amd64,
kubernetes.io/os=linux, role=web
worker-2            Ready     <none>    15m   v1.30.14   beta.kubernetes.io/arch=amd64,
kubernetes.io/os=linux, role=database
worker-3            Ready     <none>    16m   v1.30.14   beta.kubernetes.io/arch=amd64,
kubernetes.io/os=linux, role=storage
worker-4            Ready     <none>    19m   v1.30.14   beta.kubernetes.io/arch=amd64,
kubernetes.io/os=linux, role=apps
root@k8s-master:~# █

```

Figure: Node Labeling

Create Namespaces

Command	Purpose
<code>kubectl create namespace web</code>	Creates web namespace.
<code>kubectl create namespace database</code>	Creates database namespace.
<code>kubectl create namespace cache</code>	Creates cache namespace.
<code>kubectl create namespace storage</code>	Creates storage namespace.

```

root@k8s-master:~# kubectl get ns
NAME                STATUS    AGE
cache               Active    8s
database            Active    8s
default             Active    44m
kube-node-lease     Active    44m
kube-public         Active    44m
kube-system         Active    44m
metallb-system      Active    16m
storage             Active    8s
web                 Active    8s
root@k8s-master:~# █

```

Figure: Namespaces

APPLICATION DEPLOYMENT

Nginx

Command	Purpose
<code>kubectl apply -f manifests/nginx-deployment.yaml</code>	Deploys Nginx application.
<code>kubectl apply -f manifests/nginx-service.yaml</code>	Creates Nginx service.

```

root@k8s-master:~# nano nginx.yaml
root@k8s-master:~# kubectl apply -f nginx.yaml
deployment.apps/nginx created
service/nginx-service created

```

```

root@k8s-master:~# kubectl get pods -n web -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINATED NODE   READINESS GATES
nginx-66ccff7bd8-fgdkt 1/1     Running   0           31s   192.168.226.66  worker-1 <none>          <none>
nginx-66ccff7bd8-lb6dw 1/1     Running   0           31s   192.168.226.65  worker-1 <none>          <none>
root@k8s-master:~# kubectl get svc -n web
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
nginx-service       LoadBalancer 10.106.79.71    192.168.0.240 80:32643/TCP     84s

```

Figure: Nginx Deployment.

Apache

Command	Purpose
kubectl apply -f manifests/apache-deployment.yaml	Deploys Apache server.

```

root@k8s-master:~# kubectl get pods -n web -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINATED NODE
apache-8875fdb8c-btxvc 1/1     Running   0           51s   192.168.226.68  worker-1 <none>
apache-8875fdb8c-cfnkf 1/1     Running   0           51s   192.168.226.67  worker-1 <none>
nginx-66ccff7bd8-fgdkt 1/1     Running   0           9m4s   192.168.226.66  worker-1 <none>
nginx-66ccff7bd8-lb6dw 1/1     Running   0           9m4s   192.168.226.65  worker-1 <none>
root@k8s-master:~# kubectl get svc -n web
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
apache-service       LoadBalancer 10.99.101.178    192.168.0.241 80:30795/TCP     2m38s
nginx-service        LoadBalancer 10.106.79.71     192.168.0.240 80:32643/TCP     10m
root@k8s-master:~# curl 192.168.0.241
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>

```

Figure: Apache Deployment.

MySQL

Command	Purpose
kubectl apply -f manifests/mysql-pvc.yaml	Creates storage claim.
kubectl apply -f manifests/mysql-deployment.yaml	Deploys MySQL.

```

root@k8s-master:~# kubectl get pods -n database -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINATED NODE
mysql-676f465746-97jj7 1/1     Running   0           3m    192.168.133.193  worker-2 <none>
postgres-757885f655-8hv9m 1/1     Running   0           2m7s  192.168.133.194  worker-2 <none>
root@k8s-master:~#

```

Figure: MySQL Deployment.

PostgreSQL

Command	Purpose
kubectl apply -f manifests/postgres-deployment.yaml	Deploys PostgreSQL.

Redis

Command	Purpose
kubectl apply -f manifests/redis-deployment.yaml	Deploys Redis cache.

```

root@k8s-master:~# kubectl get pods -A -o wide
NAMESPACE      NAME                                     READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
cache           redis-7b8446847d-hhtx5                 1/1     Running   0           11s   192.168.38.69   worker-4         <none>            <none>
database       mysql-676f465746-97jj7                 1/1     Running   0           15m   192.168.133.193 worker-2         <none>            <none>
database       postgres-757885f655-8hv9m              1/1     Running   0           14m   192.168.133.194 worker-2         <none>            <none>
kube-system    calico-kube-controllers-5b64744bcd-65dd7 1/1     Running   0           148m   192.168.235.194 k8s-master      <none>            <none>
kube-system    calico-node-4cp69                       1/1     Running   0           142m   192.168.0.4     worker-2         <none>            <none>
kube-system    calico-node-54stq                       1/1     Running   0           140m   192.168.0.3     worker-1         <none>            <none>
kube-system    calico-node-7kfjg                       1/1     Running   0           148m   192.168.0.2     k8s-master      <none>            <none>
kube-system    calico-node-md7r5                       1/1     Running   0           143m   192.168.0.5     worker-3         <none>            <none>
kube-system    calico-node-x9hz7                       1/1     Running   0           146m   192.168.0.6     worker-4         <none>            <none>
kube-system    coredns-55cb58b774-kcsn8               1/1     Running   0           149m   192.168.235.193 k8s-master      <none>            <none>
kube-system    coredns-55cb58b774-s6tv7               1/1     Running   0           149m   192.168.235.195 k8s-master      <none>            <none>
kube-system    etcd-k8s-master                         1/1     Running   0           149m   192.168.0.2     k8s-master      <none>            <none>
kube-system    kube-apiserver-k8s-master               1/1     Running   0           149m   192.168.0.2     k8s-master      <none>            <none>
kube-system    kube-controller-manager-k8s-master      1/1     Running   0           149m   192.168.0.2     k8s-master      <none>            <none>
kube-system    kube-proxy-2nj2w                       1/1     Running   0           140m   192.168.0.3     worker-1         <none>            <none>
kube-system    kube-proxy-c4k57                       1/1     Running   0           142m   192.168.0.4     worker-2         <none>            <none>
kube-system    kube-proxy-hnnmb                       1/1     Running   0           149m   192.168.0.2     k8s-master      <none>            <none>
kube-system    kube-proxy-pmk9g                       1/1     Running   0           143m   192.168.0.5     worker-3         <none>            <none>
kube-system    kube-proxy-wvtmq                       1/1     Running   0           146m   192.168.0.6     worker-4         <none>            <none>
kube-system    kube-scheduler-k8s-master               1/1     Running   0           149m   192.168.0.2     k8s-master      <none>            <none>
metallb-system controller-86f5578878-2ldqg               1/1     Running   1 (116m ago) 118m   192.168.38.67   worker-4         <none>            <none>
metallb-system speaker-7zvzp                          1/1     Running   0           118m   192.168.0.3     worker-1         <none>            <none>
metallb-system speaker-f9fr9                          1/1     Running   0           118m   192.168.0.2     k8s-master      <none>            <none>
metallb-system speaker-kpvpk                          1/1     Running   0           118m   192.168.0.4     worker-2         <none>            <none>
metallb-system speaker-n4766                          1/1     Running   0           118m   192.168.0.6     worker-4         <none>            <none>
metallb-system speaker-v6958                          1/1     Running   0           118m   192.168.0.5     worker-3         <none>            <none>
storage        minio-65bdb94fdf-qr7tz                  1/1     Running   0           9m27s   192.168.97.193  worker-3         <none>            <none>
web            apache-8875fdbcb8-btxvc                  1/1     Running   0           19m   192.168.226.68  worker-1         <none>            <none>
web            apache-8875fdbcb8-cfnkf                  1/1     Running   0           19m   192.168.226.67  worker-1         <none>            <none>
web            nginx-66ccff7bd8-fgdkf                   1/1     Running   0           28m   192.168.226.66  worker-1         <none>            <none>
web            nginx-66ccff7bd8-lb6dw                   1/1     Running   0           28m   192.168.226.65  worker-1         <none>            <none>

```

Figure: Redis Deployment.

MinIO

Command	Purpose
kubectl apply -f manifests/minio-deployment.yaml	Deploys MinIO storage.
kubectl apply -f manifests/minio-service.yaml	Creates MinIO service.

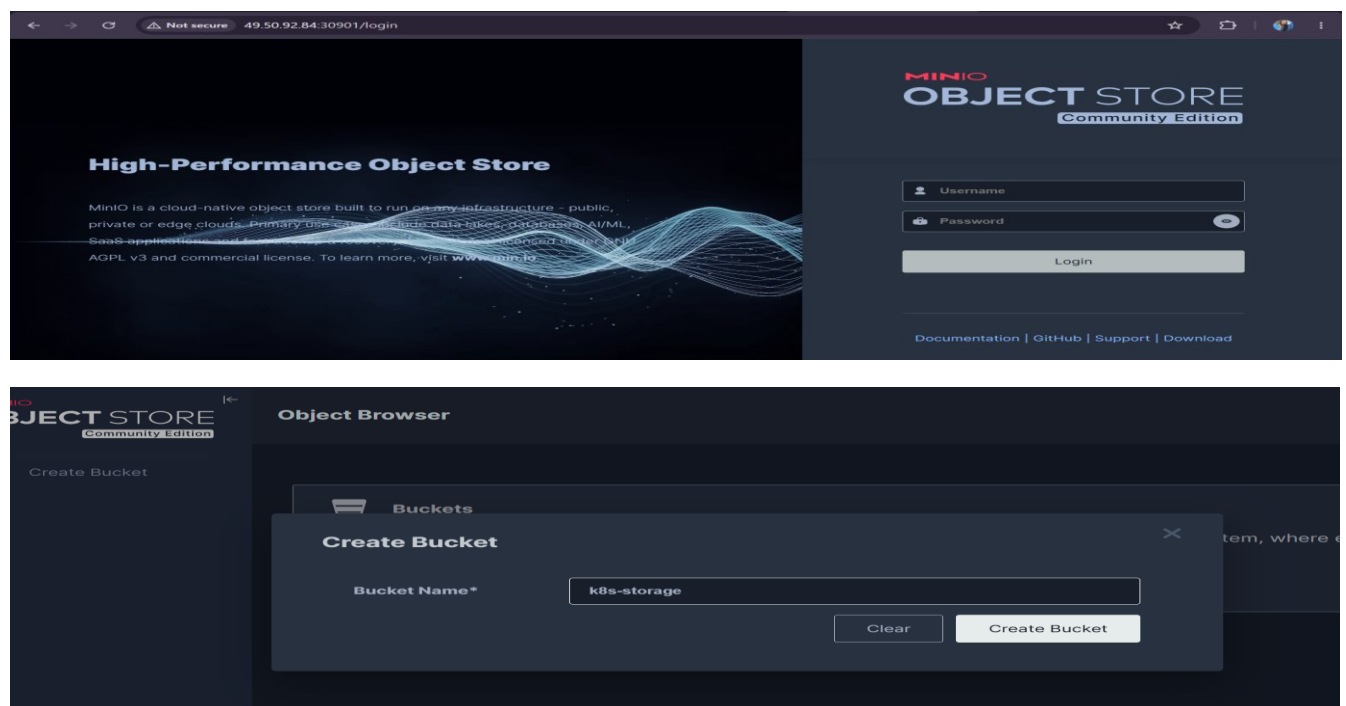


Figure: MinIO Deployment.

Scaling

Command	Purpose
kubectl scale deployment nginx --replicas=5 -n web	Increases Nginx replicas.
kubectl get pods -n web	Verifies scaling.

```
root@k8s-master:~# kubectl scale deployment nginx --replicas=5 -n web
deployment.apps/nginx scaled
root@k8s-master:~# kubectl get pods -n web -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
apache-8875fdb8c8-btxvc             1/1     Running   0           101m  192.168.226.68  worker-1         <none>            <none>
apache-8875fdb8c8-cfnkf             1/1     Running   0           101m  192.168.226.67  worker-1         <none>            <none>
nginx-66ccff7bd8-2h8tt              1/1     Running   0           112s  192.168.226.69  worker-1         <none>            <none>
nginx-66ccff7bd8-fgdkt              1/1     Running   0           109m  192.168.226.66  worker-1         <none>            <none>
nginx-66ccff7bd8-lb6dw              1/1     Running   0           109m  192.168.226.65  worker-1         <none>            <none>
nginx-66ccff7bd8-nnrhx              1/1     Running   0           112s  192.168.226.71  worker-1         <none>            <none>
nginx-66ccff7bd8-tf292              1/1     Running   0           112s  192.168.226.70  worker-1         <none>            <none>
```

Figure: Scaling

Self-Healing

Command	Purpose
kubectl delete pod POD_NAME -n web	Deletes pod manually.
kubectl get pods -n web -w	Watches automatic recreation.

```
root@k8s-master:~# kubectl delete pod nginx-66ccff7bd8-fgdkt -n web
pod "nginx-66ccff7bd8-fgdkt" deleted
root@k8s-master:~# kubectl get pods -n web -w
NAME                                READY   STATUS    RESTARTS   AGE
apache-8875fdb8c8-btxvc             1/1     Running   0           103m
apache-8875fdb8c8-cfnkf             1/1     Running   0           103m
nginx-66ccff7bd8-2h8tt              1/1     Running   0           3m54s
nginx-66ccff7bd8-lb6dw              1/1     Running   0           111m
nginx-66ccff7bd8-nnrhx              1/1     Running   0           3m54s
nginx-66ccff7bd8-qdpjl              1/1     Running   0           9s
nginx-66ccff7bd8-tf292              1/1     Running   0           3m54s
```

Figure: Self-Healing

Rolling Updates

Command	Purpose
kubectl set image deployment/nginx nginx=nginx:1.27 -n web	Updates container image.
kubectl rollout status deployment nginx -n web	Monitors update progress.

```
root@k8s-master:~# kubectl rollout undo deployment nginx -n web
deployment.apps/nginx rolled back
root@k8s-master:~#
```



```

waiting for deployment nginx rollout to finish: 3 out of 3 new replicas have
^Croot@k8s-master:~# kubectl scale deployment nginx --replicas=2 -n web
deployment.apps/nginx scaled
root@k8s-master:~# kubectl rollout undo deployment nginx -n web
deployment.apps/nginx rolled back
root@k8s-master:~# kubectl rollout status deployment nginx -n web
deployment "nginx" successfully rolled out
root@k8s-master:~#

```

Figure: Rolling Updates

Rollback

Command	Purpose
<code>kubectl rollout undo deployment nginx -n web</code>	Restores previous version.

```

root@k8s-master:~# kubectl rollout undo deployment nginx -n web
deployment.apps/nginx rolled back
root@k8s-master:~#

```

Figure: Rollback

Persistent Storage

Command	Purpose
<code>kubectl get pv</code>	Displays Persistent Volumes.
<code>kubectl get pvc -n database</code>	Displays Persistent Volume Claims.

```

root@k8s-master:~# nano mysql-pv.yaml
root@k8s-master:~# kubectl apply -f mysql-pv.yaml
persistentvolume/mysql-pv created
root@k8s-master:~# kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM  STORAGECLASS  VOLUMEATTRIBUTESCLASS
mysql-pv      5Gi       RWO           Retain          Available    


```

```

root@k8s-master:~# nano mysql-pvc.yaml
root@k8s-master:~# kubectl apply -f mysql-pvc.yaml
persistentvolumeclaim/mysql-pvc created
root@k8s-master:~# kubectl get pvc -n database
NAME          STATUS    VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS
mysql-pvc     Bound     mysql-pv        5Gi       RWO             


```

```

kubectl get pvc -n database
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS
mysql-pv      5Gi       RWO           Retain          Bound     database/mysql-pvc     

NAME          STATUS    VOLUME          CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
mysql-pvc     Bound     mysql-pv        5Gi       RWO           <unset>        7m53s
root@k8s-master:~#

```

Figure: Persistent storage

CONFIGMAP

Command	Purpose
<code>kubectl create configmap website-content --from-file=index.html -n web</code>	Creates ConfigMap.
<code>kubectl rollout restart deployment nginx -n web</code>	Applies updated website content.



Figure: Website Deployment.

Metrics Server

Command	Purpose
kubectl top nodes	Displays node resource usage.
kubectl top pods -A	Displays pod resource usage.

```

root@k8s-master:~# kubectl top nodes
NAME          CPU (cores)   CPU%   MEMORY(bytes)   MEMORY%
k8s-master    71m           1%     3103Mi          31%
worker-1      40m           1%     2366Mi          24%
worker-2      73m           1%     3089Mi          31%
worker-3      35m           0%     2280Mi          23%
worker-4      53m           1%     2326Mi          23%

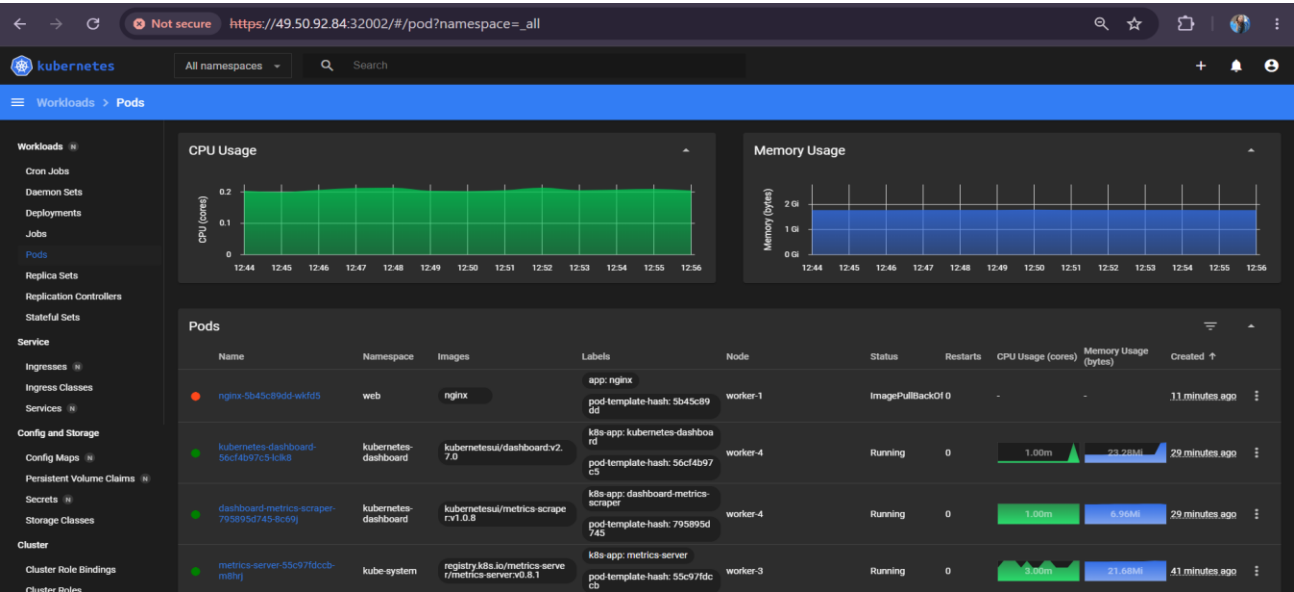
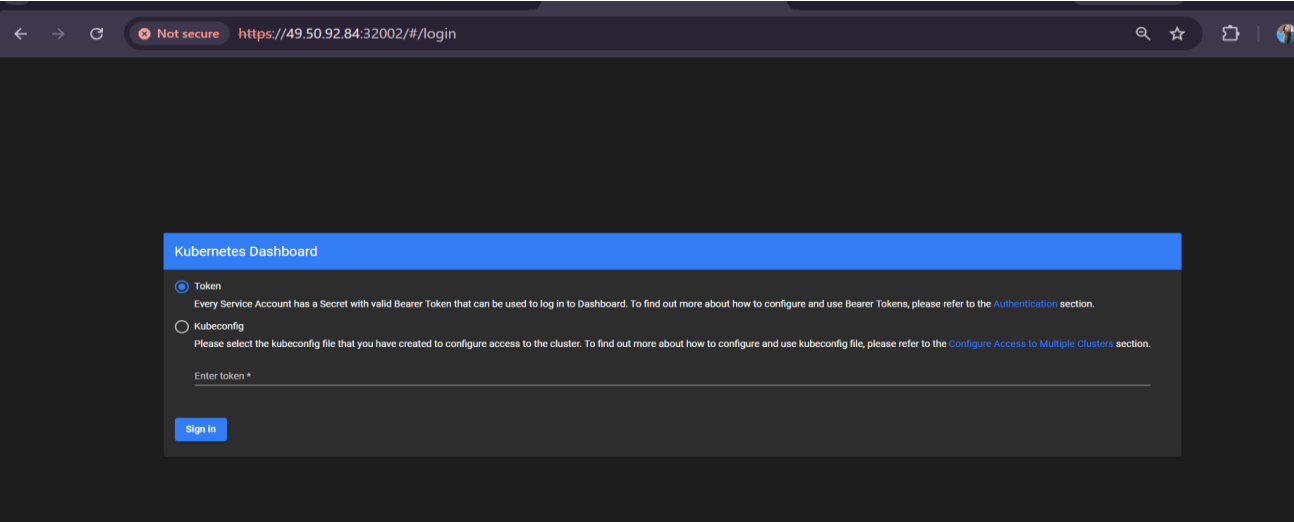
root@k8s-master:~# kubectl top pods -A
NAMESPACE     NAME                                                    CPU (cores)   MEMORY (bytes)
cache         redis-7b8446847d-hhtx5                                3m            5Mi
database      mysql-7f5569f55d-gnl1nn                               5m            390Mi
database      postgres-757885f655-8hv9m                             1m            31Mi
kube-system   calico-kube-controllers-5b64744bcd-65dd7              4m            27Mi
kube-system   calico-node-4cp69                                     29m           135Mi
kube-system   calico-node-54stq                                     32m           135Mi
kube-system   calico-node-7kfjg                                     17m           136Mi
kube-system   calico-node-md7r5                                     17m           135Mi
kube-system   calico-node-x9hz7                                     31m           136Mi
kube-system   coredns-55cb58b774-kcsn8                             1m            14Mi
kube-system   coredns-55cb58b774-s6tv7                             1m            14Mi
kube-system   etcd-k8s-master                                       15m           58Mi
kube-system   kube-apiserver-k8s-master                             35m          293Mi
kube-system   kube-controller-manager-k8s-master                   8m            56Mi
kube-system   kube-proxy-2nj2w                                     1m            13Mi
kube-system   kube-proxy-c4k57                                     1m            12Mi
kube-system   kube-proxy-hnnmb                                     1m            13Mi
kube-system   kube-proxy-pmk9g                                     1m            12Mi
kube-system   kube-proxy-wvtmq                                     1m            12Mi
kube-system   kube-scheduler-k8s-master                             2m            19Mi
kube-system   metrics-server-55c97fdccb-m8hrj                     2m            22Mi
kubernetes-dashboard dashboard-metrics-scraper-795895d745-8c69j             1m             8Mi
kubernetes-dashboard kubernetes-dashboard-56cf4b97c5-1cl1k8              1m            11Mi
storage       minio-694b84fdbc-gcbfj                               1m           102Mi
web           apache-8875fdbbc8-btxvc                              1m             7Mi
web           apache-8875fdbbc8-cfnkf                              1m             8Mi
web           nginx-5bcbf5f8d6-7wff4                              0m             4Mi
web           nginx-5bcbf5f8d6-c4s79                              0m             4Mi

```

Figure: Metrics server

Kubernetes Dashboard

Command	Purpose
kubectl get svc -n kubernetes-dashboard	Displays dashboard service.
https://PUBLIC-IP:32002	Accesses dashboard GUI.

A screenshot of the Kubernetes Dashboard showing Persistent Volume Claims. The top navigation bar shows 'Config And Storage > Persistent Volume Claims'. The left sidebar has a menu with 'Workloads', 'Pods', 'Replica Sets', 'Replication Controllers', 'Service', 'Ingresses', 'Ingress Classes', 'Services', 'Config and Storage', 'Config Maps', 'Persistent Volume Claims', 'Secrets', 'Storage Classes', 'Cluster', 'Cluster Role Bindings', and 'Cluster Roles'. The main content area shows a table of Persistent Volume Claims. The table has columns: Name, Namespace, Labels, Status, Volume, Capacity, Access Modes, Storage Class, and Created. The table shows one claim: 'mysql-pvc' (database, -, Bound, mysql-pv, 5Gi, ReadWriteOnce, -, 2 days ago).

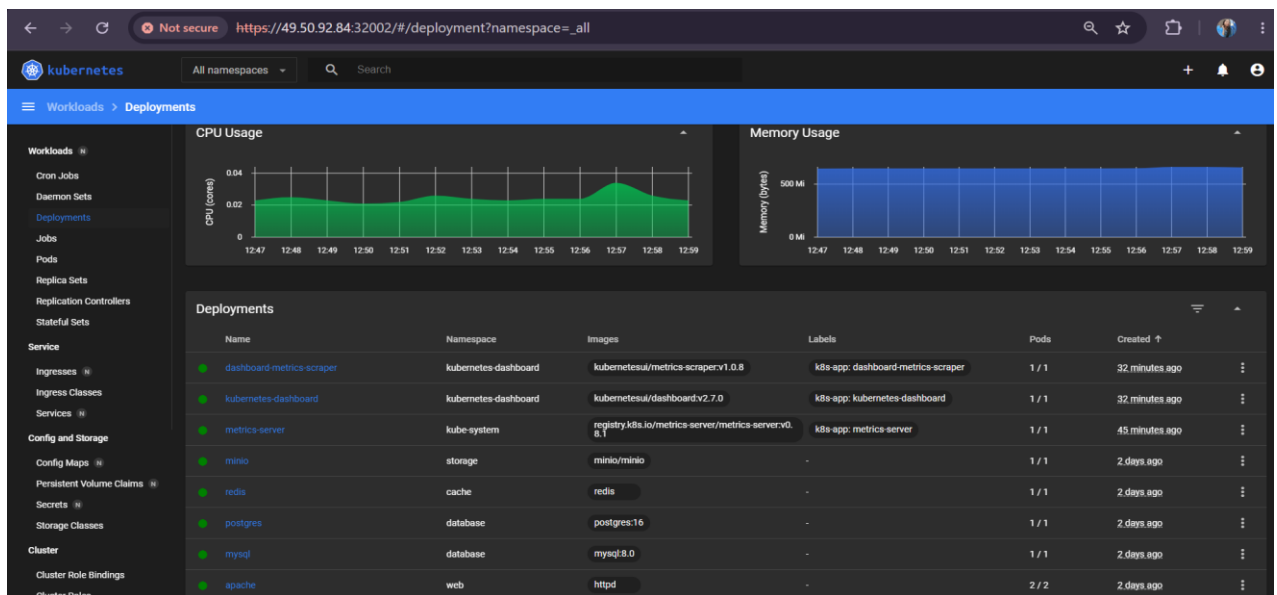


Figure: Kubernetes dashboard

Challenges Encountered

Issue	Resolution
Metrics Server ImagePullBackOff	Changed image source
Dashboard inaccessible	Changed service to NodePort
Git push rejected	Pulled remote repository
ConfigMap errors	Corrected YAML indentation
Rollout freeze	Reduced replicas
MinIO inaccessible	Configured DNAT

Table: Challenges Encountered

GitHub Repository

<https://github.com/2121-shivi/Kubernetes-Enterprise-Cluster>

Conclusion

The Kubernetes Enterprise Cluster was successfully designed and deployed on VMware Cloud Director using Ubuntu Linux and kubeadm. The implementation validated key Kubernetes capabilities, including multi-node cluster management, container orchestration, persistent and object storage, self-healing, rolling updates, resource monitoring, dashboard management, and high availability. Through this project, practical experience was gained in deploying and administering enterprise-grade Kubernetes environments, strengthening knowledge of cloud-native infrastructure, containerized application deployment, and modern DevOps practices. This implementation provides a strong foundation for managing scalable, reliable, and production-ready Kubernetes clusters in enterprise environments.